

Product Requirement Document (PRD)

Project Overview: Tech Job Aggregator & Analytics Hub

- **Document Version:** 1.0.0
- **Target Audience:** Software engineers, tech professionals, and data-driven job seekers.
- **Objective:** To build a centralized, real-time job aggregation platform that removes the friction of jumping between multiple job boards. By combining high-frequency scraping with deep market analytics and instant alert systems, the platform delivers an elite job-hunting experience while acting as a showcase of production-ready full-stack architecture.

1. Core Value Proposition

1. **Aggregation without Friction:** Consolidate fragmented job listings into a single, highly searchable interface requiring zero initial user commitment.
2. **Market Intelligence:** Transform raw job descriptions into structured data points to provide transparent insights into technology trends and demand.
3. **First-Mover Advantage:** Give premium users an edge in a competitive market by delivering instant notifications the moment a matching role is published anywhere on the web.

2. User Matrix & Monetization Tiers

Feature	Base Plan (Free)	Pro Plan (Subscription)
Authentication	Not required for basic browsing.	Required (OAuth2 / Email-Password).
Search & Filtering	Full access to keyword search and filters.	Full access + saved search presets.
Job Discovery	View listings aggregated up to the current day.	View listings + priority access.

Analytics Engine	Basic macro trends (Top technologies, aggregate numbers).	Deep analytics (Tech correlations, historical trends).
Notification Engine	None. Manual site checking only.	Instant Telegram webhooks and Email digests.

3. Functional Requirements

3.1. Aggregation & Data Pipeline (System Level)

The platform must run an independent background pipeline to ingest, clean, and standardize data from external job boards without degrading the performance of the user dashboard.

- **Scraping Engine:** Autonomous cron-based workers that pull data from target platforms (e.g., Jobinja, Jobvision, Quera).
- **Data Normalization:** A pipeline that intercepts varied raw payloads and maps them to a unified schema (standardizing fields like **title**, **company**, **location**, **salary_range**, **experience_level**, and **posted_at**).
- **Skill Extraction:** An internal parser that extracts specific technical keywords (e.g., React, Node.js, Docker, Python) from raw description text and saves them as standardized relational tags.
- **Deduplication:** A hashing mechanism (based on company name, normalized title, and posting date) to ensure identical listings across different platforms are merged into a single record.

3.2. Base Plan Features (Free Tier)

Designed for low-friction user acquisition, allowing immediate utility upon landing on the site.

- **Unified Search Engine:** Full-text search querying job titles, companies, and descriptions.
- **Advanced Filtering Layout:**
 - Filter by **Category** (Frontend, Backend, DevOps, Data Science, etc.).
 - Filter by **Work Type** (Remote, Hybrid, On-site).
 - Filter by **City/Location**.
- **Basic Analytics Dashboard:** A public dashboard presenting high-level data visualisations:
 - **Volume Tracking:** Total active tech job openings over the last 30 days.
 - **Tech Leaderboard:** A simple bar chart displaying the Top 10 most demanded technologies/tags across the entire market.

3.3. Pro Plan Features (Premium Tier)

High-value automated features focused on speed and deep analytical insight.

- **Instant Alert Configuration Engine:**
 - Users can create granular search queries (e.g., "React Developer, Remote or Tehran, Mid-Senior").
 - Users can tie these queries to active delivery channels.
- **Telegram Delivery Hook:** Integration via a dedicated Telegram bot. The system pushes a cleanly formatted message containing the job details and a direct application link within 5 minutes of data normalization.
- **Email Digest System:** Daily or weekly automated HTML emails summarizing newly scraped positions matching the user's criteria.
- **Advanced Analytics Engine:**
 - **Technology Co-occurrence:** Insights revealing complementary skills (e.g., "Companies hiring for NestJS look for TypeScript 90% of the time and Docker 65% of the time").
 - **Market Share Shift:** Month-over-month percentage changes in framework popularity (e.g., Vue vs. React trendlines).
 - **Salary Distribution:** Aggregated salary curve visualizations plotted against experience levels and locations (derived from listings containing transparent compensation data).

4. Technical Architecture Requirements

To maximize system maintainability and ensure the project serves as an elite portfolio piece, the code base must implement rigid architectural boundaries.

4.1. Service Separation

- **Scraper Microservice:** Isolated node/script environment responsible solely for data extraction, rate-limiting management, and database ingestion.
- **Core API Application:** A secure REST or GraphQL framework handling user state, authentication, alert settings, and system queries.
- **Frontend Client:** A highly optimized single-page or server-rendered application focused on layout speed, filter snappiness, and fluid data visualizations.

4.2. Database Architecture & Optimization

- **Relational Storage:** A structured database (e.g., PostgreSQL) leveraging explicit foreign key constraints to manage relationships between Users, Subscriptions, Jobs, Tags, and Active Alerts.
- **Index Strategy:** Critical columns used frequently by the search engine (**tags**, **category**, **posted_at**) must be indexed to guarantee sub-100ms response times as historical records scale.
- **Cache Strategy:** Global analytics queries must be cached (e.g., using Redis or internal application memory) and refreshed on a strict schedule rather than computing on every page load.

4.3. Infrastructure & Deployment

- **Containerization:** The entire application stack (API, database, background workers, client) must be fully containerized using standard configuration files ([Docker](#) / [Docker Compose](#)).
- **Production Portability:** The stack must run reliably on a standard Linux environment using modest hardware allocations, demonstrating clean resource isolation.

5. Non-Functional Requirements

- **Security:** Subscription endpoints and premium analytics routes must be strictly protected behind verified authentication middleware. Sensitive information, such as passwords, must be hashed before storage.
- **Scraper Resiliency:** The data aggregation service must catch connection errors, handle external structural updates gracefully without crashing the core app, and rotate timing signatures to minimize IP blockages.
- **Data Integrity:** Old or expired job listings should not be casually deleted; they must be flagged as inactive to preserve historical data for the analytics engine.